

# 熊猫API框架

## 完整功能 Demo 说明书

PandaAPI Framework - Complete Demo Guide

版本: v1.0

日期: 2024年

# 目录

第1章 框架概述

第2章 项目结构

第3章 PandaDB 数据库操作

第4章 缓存系统

第5章 路由系统

第6章 验证器

第7章 中间件

第8章 公共函数

第9章 MVC 应用开发

第10章 API 接口测试

# 第1章 框架概述

## 1.1 框架简介

熊猫API框架 (PandaAPI) 是一个轻量级、高性能的 PHP API 开发框架，融合了 ThinkPHP 和 PandaDB 两种数据库操作风格，内置缓存系统、路由引擎、验证器、中间件等完整功能，旨在帮助开发者快速构建 RESTful API 应用。

框架遵循简洁优雅的设计理念，提供直观的链式调用 API，同时保持极低的性能开销，适合中小型 API 项目快速开发。

## 1.2 技术优势

| 序号 | 特性       | 说明                                     |
|----|----------|----------------------------------------|
| 1  | 双风格数据库操作 | 同时支持 ThinkPHP 链式查询和 PandaDB 快捷方法       |
| 2  | 多缓存驱动    | 支持 File / Redis / Memcache / OpenResty |
| 3  | 灵活路由系统   | 支持 RESTful 路由、路由组、参数约束                 |
| 4  | 内置验证器    | 提供丰富的验证规则，支持自定义扩展                      |
| 5  | 中间件机制    | 支持全局中间件和路由级中间件                         |
| 6  | MVC 架构   | 支持控制器/模型分层，也支持闭包路由快速开发                 |
| 7  | 事务支持     | 提供回调式和手动式两种事务操作方式                      |
| 8  | SQL 日志   | 内置慢查询检测和 SQL 执行日志                      |

## 1.3 环境要求

- PHP >= 7.4 (推荐 8.0+)
- PDO PHP 扩展 (必需)
- Redis 扩展 (可选，用于 Redis 缓存驱动)
- Composer (用于依赖管理)
- Web 服务器：Apache / Nginx / PHP 内置服务器

## 1.4 快速启动

快速启动命令

```
# 1. 克隆项目
git clone https://github.com/pandaapi/pandaapi-demo.git
cd pandaapi-demo

# 2. 安装依赖
composer install

# 3. 配置数据库 (编辑 config/database.php)
# 4. 初始化数据库
<?php init_database.<?php
```

# 5. 启动开发服务器

```
<?php -S localhost:8080 -t public
```

# 6. 访问测试

```
curl http://localhost:8080/api/v1/health
```

## 第2章 项目结构

### 2.1 目录结构

#### 项目目录结构

```
pandaapi/
+-- app/                # 应用目录
| +-- Controllers/      # 控制器目录
| | +-- UserController.<?php
| | +-- PostController.<?php
| +-- Models/           # 模型目录
| | +-- UserModel.<?php
| | +-- PostModel.<?php
| +-- common.<?php       # 公共函数文件
+-- config/              # 配置目录
| +-- database.<?php     # 数据库配置
| +-- cache.<?php        # 缓存配置
| +-- app.<?php          # 应用配置
+-- public/              # Web 根目录
| +-- index.<?php        # 入口文件
+-- routes/              # 路由目录
| +-- api.<?php           # API 路由定义
| +-- web.<?php           # Web 路由定义
+-- runtime/             # 运行时目录
| +-- cache/             # 缓存文件
| +-- logs/              # 日志文件
+-- vendor/              # Composer 依赖
+-- composer.json        # Composer 配置
+-- init_database.<?php   # 数据库初始化脚本
```

### 2.2 MVC 架构说明

框架采用经典的 MVC (Model-View-Controller) 架构模式：

Model (模型)：负责数据库交互和数据逻辑，继承 PandaDB 的链式查询能力

View (视图)：API 框架中视图层由 JSON 响应替代，通过 Response 类统一输出

Controller (控制器)：处理请求逻辑，调用模型获取数据，返回响应

同时，框架也支持闭包路由模式，适合小型项目快速开发，无需创建控制器文件。

### 2.3 请求生命周期

#### 请求生命周期

```
// 请求生命周期流程：
// 1. 用户请求 -> public/index.php
// 2. 加载 Composer 自动加载器
// 3. 创建 App 实例，加载配置
// 4. 执行全局中间件 (CORS、日志等)
// 5. 路由匹配 -> 找到对应处理函数/控制器
// 6. 执行路由级中间件 (Auth、Throttle等)
// 7. 执行业务逻辑 (控制器方法 / 闭包函数)
// 8. 返回 JSON 响应
// 9. 记录日志、发送响应

// 入口文件示例：
```

```
// public/index.php
require_once __DIR__ . '/../vendor/autoload.php';

$app = new PandaAPI\Core\App();
$app->run();
```

## 第3章 PandaDB 数据库操作

### 3.1 数据库配置

#### 数据库配置

```
<?php
use PandaAPI\Database\PandaDB;

// 数据库连接配置
PandaDB::config([
    'driver' => 'mysql',      // 数据库类型
    'host'    => '127.0.0.1', // 数据库主机
    'port'    => 3306,        // 端口
    'database' => 'pandaapi_demo', // 数据库名
    'username' => 'root',      // 用户名
    'password' => '',          // 密码
    'charset' => 'utf8mb4',    // 字符集
    'prefix' => 'panda_',      // 表前缀
]);
```

### 3.2 ThinkPHP 风格链式查询

#### ThinkPHP 风格链式查询

```
<?php
use PandaAPI\Database\PandaDB;

// 查询多条记录 (select)
$users = PandaDB::table('users')
    ->where('status', 1)
    ->order('created_at', 'desc')
    ->limit(10)
    ->select();

// 查询单条记录 (find)
$user = PandaDB::table('users')
    ->where('id', 1)
    ->find();

// 多条件查询
$users = PandaDB::table('users')
    ->where('status', 1)
    ->where('age', '>=', 18)
    ->order('id', 'asc')
    ->limit(0, 20)
    ->select();

// 指定字段
$users = PandaDB::table('users')
    ->field('id, name, email')
    ->where('status', 1)
    ->select();
```

### 3.3 PandaDB 风格快捷方法

#### PandaDB 风格快捷方法

```

<?php
use PandaAPI\Database\PandaDB;

// 查询多条 (select)
$users = PandaDB::select('users', '*', [
    'status' => 1,
    'ORDER' => ['created_at' => 'DESC'],
    'LIMIT' => 10,
]);

// 查询单条 (get)
$user = PandaDB::get('users', ['id', 'name', 'email'], [
    'id' => 1,
]);

// 插入数据 (insert)
$id = PandaDB::insert('users', [
    'name' => '张三',
    'email' => 'zhangsan@example.com',
    'password' => password_hash('123456', PASSWORD_DEFAULT),
    'status' => 1,
    'created_at' => date('Y-m-d H:i:s'),
]);

// 更新数据 (update)
PandaDB::update('users', [
    'name' => '张三',
    'updated_at' => date('Y-m-d H:i:s'),
], ['id' => 1]);

// 删除数据 (delete)
PandaDB::delete('users', ['id' => 1]);

```

### 3.4 聚合查询

#### 聚合查询

```

<?php
use PandaAPI\Database\PandaDB;

// count - 计数
$total = PandaDB::table('users')->count();
$active = PandaDB::table('users')->where('status', 1)->count();

// sum - 求和
$totalViews = PandaDB::table('posts')->sum('views');

// avg - 平均值
$avgViews = PandaDB::table('posts')->avg('views');

// max - 最大值
$maxViews = PandaDB::table('posts')->max('views');

// min - 最小值
$minViews = PandaDB::table('posts')->min('views');

// 综合统计
$stats = [
    'total_users' => PandaDB::table('users')->count(),
    'total_posts' => PandaDB::table('posts')->count(),
    'total_views' => PandaDB::table('posts')->sum('views'),

```



```
'avg_views' => round(PandaDB::table('posts')->avg('views'), 2),  
];
```

## 3.5 JOIN 关联查询

### JOIN 关联查询

```
<?php  
use PandaAPI\Database\PandaDB;  
  
// INNER JOIN 查询  
$posts = PandaDB::table('posts')  
->field('posts.*, users.name as author_name')  
->join('users', 'posts.user_id = users.id')  
->where('posts.status', 1)  
->order('posts.created_at', 'desc')  
->select();  
  
// LEFT JOIN 查询  
$posts = PandaDB::table('posts AS p')  
->field('p.id, p.title, p.views, u.name as author_name')  
->leftJoin('users AS u', 'p.user_id = u.id')  
->where('p.status', 1)  
->select();  
  
// 多表 JOIN  
$result = PandaDB::table('orders AS o')  
->field('o.*, u.name, p.title')  
->join('users AS u', 'o.user_id = u.id')  
->join('products AS p', 'o.product_id = p.id')  
->where('o.status', 'completed')  
->select();
```

## 3.6 分页查询

### 分页查询

```
<?php  
use PandaAPI\Database\PandaDB;  
  
// paginate 分页查询  
$page = (int)($_GET['page'] ?? 1);  
$perPage = (int)($_GET['per_page'] ?? 15);  
  
$result = PandaDB::table('users')  
->where('status', 1)  
->order('created_at', 'desc')  
->paginate($perPage, $page);  
  
// 返回结构：  
// $result = [  
//   'data' => [...], // 当前页数据  
//   'total' => 100, // 总记录数  
//   'current_page' => 1, // 当前页码  
//   'per_page' => 15, // 每页条数  
//   'last_page' => 7, // 最后一页  
// ];  
  
// 配合 Response::paginate 返回 JSON  
return Response::paginate(
```

```
$result['data'],
$result['total'],
$result['current_page'],
$result['per_page']
);
```

## 3.7 WHERE 高级条件

### WHERE 高级条件

```
<?php
use PandaAPI\Database\PandaDB;

// whereIn - IN 查询
$users = PandaDB::table('users')
    ->whereIn('id', [1, 2, 3, 4, 5])
    ->select();

// whereNotIn - NOT IN 查询
$users = PandaDB::table('users')
    ->whereNotIn('status', [0, -1])
    ->select();

// whereBetween - 范围查询
$users = PandaDB::table('users')
    ->whereBetween('age', [18, 60])
    ->select();

// whereLike - 模糊查询
$users = PandaDB::table('users')
    ->whereLike('name', '%a%')
    ->select();

// whereNull - NULL 查询
$users = PandaDB::table('users')
    ->whereNull('deleted_at')
    ->select();

// whereNotNull - NOT NULL 查询
$users = PandaDB::table('users')
    ->whereNotNull('email')
    ->select();

// whereOr - OR 条件
$users = PandaDB::table('users')
    ->where('status', 1)
    ->orWhere('role', 'admin')
    ->select();

// whereRaw - 原生条件
$users = PandaDB::table('users')
    ->whereRaw("DATE(created_at) = '2024-01-01'")
    ->select();
```

## 3.8 原生 SQL

### 原生 SQL

```
<?php
use PandaAPI\Database\PandaDB;
```

```

// query - 原生查询 (带参数绑定, 防SQL注入)
$users = PandaDB::query(
    "SELECT * FROM panda_users WHERE status = ? ORDER BY id DESC LIMIT ?",
    [1, 10]
);

// execute - 原生执行 (INSERT/UPDATE/DELETE)
PandaDB::execute(
    "UPDATE panda_users SET status = ? WHERE id = ?",
    [0, 5]
);

// 原生聚合查询
$result = PandaDB::query(
    "SELECT COUNT(*) as total, AVG(views) as avg_views FROM panda_posts"
);

// 复杂原生查询
$stats = PandaDB::query(
    "SELECT u.name, COUNT(p.id) as post_count
    FROM panda_users u
    LEFT JOIN panda_posts p ON u.id = p.user_id
    GROUP BY u.id
    HAVING post_count > 0
    ORDER BY post_count DESC"
);

```

### 3.9 事务操作

#### 事务操作

```

<?php
use PandaAPI\Database\PandaDB;

// 方式一: 回调式事务 (推荐)
try {
    $result = PandaDB::transaction(function() {
        $userId = PandaDB::table('users')->insert([
            'name' => 'xxxxxx',
            'email' => 'tx_' . time() . '@test.com',
            'password' => password_hash('123456', PASSWORD_DEFAULT),
            'status' => 1,
            'created_at' => date('Y-m-d H:i:s'),
        ]);

        PandaDB::table('posts')->insert([
            'user_id' => $userId,
            'title' => 'xxxx',
            'content' => 'xxxxxxxxxx',
            'status' => 1,
            'created_at' => date('Y-m-d H:i:s'),
        ]);

        return ['user_id' => $userId];
    });

    // 事务自动提交
} catch (Exception $e) {
    // 事务自动回滚
}

```

```

        echo '□□□□: ' . $e->getMessage();
    }

// 方式二：手动事务
PandaDB::beginTransaction();
try {
    PandaDB::table('users')->insert([...]);
    PandaDB::table('posts')->insert([...]);
    PandaDB::commit();
} catch (Exception $e) {
    PandaDB::rollBack();
}

```

### 3.10 慢查询与 SQL 日志

#### 慢查询与 SQL 日志

```

<?php
use PandaAPI\Database\PandaDB;

// 设置慢查询阈值 (毫秒)
PandaDB::setSlowThreshold(100);

// 获取查询次数
$queryCount = PandaDB::getQueryCount();

// 获取 SQL 日志
$logs = PandaDB::getQueryLog();

// 在健康检查接口中使用
Route::get('/api/v1/health', function() {
    return Response::success([
        'framework' => 'PandaAPI',
        'status'     => 'healthy',
        'db_queries' => PandaDB::getQueryCount(),
        'query_log'  => PandaDB::getQueryLog(),
    ]);
});

// SQL 日志输出示例：
// [
//   ['sql' => 'SELECT * FROM panda_users WHERE id = ?', 'time' => 1.23],
//   ['sql' => 'SELECT * FROM panda_posts ...', 'time' => 150.5], // 慢查询!
// ]

```

## 第4章 缓存系统

### 4.1 缓存配置

#### 缓存配置

```
<?php
use PandaAPI\Cache\Cache;

// 文件缓存配置
Cache::config([
    'driver' => 'file',
    'file' => [
        'path' => __DIR__ . '/../runtime/cache',
    ],
    'default_ttl' => 3600,
]);

// Redis 缓存配置
Cache::config([
    'driver' => 'redis',
    'redis' => [
        'host'      => '127.0.0.1',
        'port'      => 6379,
        'password' => null,
        'database' => 0,
        'prefix' => 'pandaapi:',
    ],
    'default_ttl' => 3600,
]);
```

### 4.2 基础操作

#### 缓存基础操作

```
<?php
use PandaAPI\Cache\Cache;

// set - 设置缓存 (带过期时间, 单位秒)
Cache::set('user:1', ['id' => 1, 'name' => '张三'], 3600);
Cache::set('config:site', ['title' => '测试'], 7200);

// get - 获取缓存
$user = Cache::get('user:1');
// 返回: ['id' => 1, 'name' => '张三'] 或 null

// get 带默认值
$user = Cache::get('user:999', ['id' => 0, 'name' => '']);

// has - 检查缓存是否存在
if (Cache::has('user:1')) {
    $user = Cache::get('user:1');
}

// delete / forget - 删除缓存
Cache::delete('user:1');
Cache::forget('user:1');
```

```
// clear - 清空所有缓存
Cache::clear();
```

## 4.3 缓存回源 remember

### 缓存回源 remember

```
<?php
use PandaAPI\Cache\Cache;
use PandaAPI\Database\PandaDB;

// Cache::remember - 缓存回源模式
// 如果缓存存在则直接返回，不存在则执行回调并自动缓存结果
$user = Cache::remember("user:{id}", function() use ($id) {
    return PandaDB::table('users')
        ->where('id', (int)$id)
        ->find();
}, 300); // 缓存 5 分钟

// 典型应用场景：用户详情接口
Route::get('/users/{id}', function($id) {
    $user = Cache::remember("user:{id}", function() use ($id) {
        return PandaDB::table('users')
            ->where('id', (int)$id)
            ->find();
    }, 300);

    if (!$user) {
        return Response::error('aaaaa', 404);
    }

    return Response::success($user);
});
```

## 4.4 原子操作 increment/decrement

### 原子操作 increment/decrement

```
<?php
use PandaAPI\Cache\Cache;

// increment - 原子递增
Cache::set('page:views', 0);
$views = Cache::increment('page:views'); // 1
$views = Cache::increment('page:views'); // 2
$views = Cache::increment('page:views', 5); // 7 (步长为5)

// decrement - 原子递减
$views = Cache::decrement('page:views'); // 6
$views = Cache::decrement('page:views', 3); // 3 (步长为3)

// 应用场景：文章浏览量计数
Route::get('/posts/{id}', function($id) {
    $cacheKey = "post:views:{id}";
    $views = Cache::increment($cacheKey);

    // 定期同步到数据库（或使用队列异步同步）
    if ($views % 100 === 0) {
        PandaDB::table('posts')
            ->where('id', (int)$id)
```

```

        ->update(['views' => $views]);
    }

    return Response::success(['views' => $views]);
});

```

## 4.5 批量操作

### 批量操作

```

<?php
use PandaAPI\Cache\Cache;

// setMultiple - 批量设置
Cache::setMultiple([
    'config:site_name' => '熊猫API',
    'config:site_desc' => '熊猫PHP',
    'config:site_version' => '1.0.0',
], 3600);

// getMultiple - 批量获取
$values = Cache::getMultiple([
    'config:site_name',
    'config:site_desc',
    'config:site_version',
]);
// 返回: ['config:site_name' => '熊猫API', ...]

// deleteMultiple - 批量删除
Cache::deleteMultiple([
    'config:site_name',
    'config:site_desc',
]);

```

## 4.6 标签缓存

### 标签缓存

```

<?php
use PandaAPI\Cache\Cache;

// 标签缓存 - 按标签分组管理缓存
// 设置带标签的缓存
Cache::set('user:1', $user1, 3600)->tag('users');
Cache::set('user:2', $user2, 3600)->tag('users');
Cache::set('user:3', $user3, 3600)->tag('users');

// 也可以设置多个标签
Cache::set('post:1', $post1, 3600)->tag('posts', 'user:1_posts');

// 按标签清除缓存
Cache::flushTag('users'); // 清除所有 users 标签的缓存
Cache::flushTag('posts'); // 清除所有 posts 标签的缓存
Cache::flushTag('user:1_posts'); // 清除指定用户的文章缓存

// 应用场景：用户信息更新时批量清除缓存
Route::put('/users/{id}', function($id) {
    // ... 更新用户数据 ...

    // 清除该用户相关的所有缓存

```

```
Cache::forget("user:${id}");  
Cache::flushTag("user:${id}_related");  
  
return Response::success(['message' => '成功']);  
});
```



## 第5章 路由系统

### 5.1 路由定义

#### 路由定义

```
<?php
use PandaAPI\Route\Route;

// GET 请求
Route::get('/api/v1/users', function() {
    return Response::success(PandaDB::table('users')->select());
});

// POST 请求
Route::post('/api/v1/users', function() {
    $data = (new Request())->all();
    $id = PandaDB::table('users')->insert($data);
    return Response::success(['id' => $id], 201);
});

// PUT 请求
Route::put('/api/v1/users/{id}', function($id) {
    $data = (new Request())->all();
    PandaDB::table('users')->where('id', $id)->update($data);
    return Response::success(['message' => '成功']);
});

// DELETE 请求
Route::delete('/api/v1/users/{id}', function($id) {
    PandaDB::table('users')->where('id', $id)->delete();
    return Response::success(['message' => '成功']);
});

// PATCH 请求
Route::patch('/api/v1/users/{id}/status', function($id) {
    $data = (new Request())->all();
    PandaDB::table('users')->where('id', $id)->update($data);
    return Response::success(['message' => '成功']);
});

// match - 匹配多种请求方法
Route::match(['GET', 'POST'], '/api/v1/search', function() {
    // 处理搜索逻辑
});

// any - 匹配所有请求方法
Route::any('/api/v1/test', function() {
    return Response::success(['message' => '成功']);
});
```

### 5.2 路由组

#### 路由组

```
<?php
use PandaAPI\Route\Route;
```

```
// 路由组 - 统一前缀和中间件
Route::group([
    'prefix'      => '/api/v1',
    'middleware' => ['Cors'],
], function() {

    // 公开接口
    Route::get('/health', [HealthController::class, 'index']);
    Route::post('/login', [AuthController::class, 'login']);
    Route::post('/register', [AuthController::class, 'register']);

    // 需要认证的接口
    Route::group(['middleware' => ['Auth']], function() {
        Route::get('/users', [UserController::class, 'index']);
        Route::get('/users/{id}', [UserController::class, 'show']);
        Route::post('/users', [UserController::class, 'store']);
        Route::put('/users/{id}', [UserController::class, 'update']);
        Route::delete('/users/{id}', [UserController::class, 'destroy']);

        Route::get('/posts', [PostController::class, 'index']);
        Route::post('/posts', [PostController::class, 'store']);
    });

    // 需要限流的接口
    Route::group(['middleware' => ['Throttle:60,1']], function() {
        Route::post('/sms/send', [SmsController::class, 'send']);
        Route::post('/email/send', [EmailController::class, 'send']);
    });
});
```

## 5.3 路由参数约束

### 路由参数约束

```
<?php
use PandaAPI\Route\Route;

// whereNumber - 数字约束
Route::get('/users/{id}', function($id) {
    // $id 只能是数字
})->whereNumber('id');

// whereAlpha - 字母约束
Route::get('/categories/{slug}', function($slug) {
    // $slug 只能是字母
})->whereAlpha('slug');

// where - 正则约束
Route::get('/posts/{year}/{month}', function($year, $month) {
    // year: 4位数字, month: 2位数字
})->where('year', '[0-9]{4}')->where('month', '[0-9]{2}');

// 组合约束
Route::get('/files/{id}/{name}', function($id, $name) {
    //
})->whereNumber('id')->where('name', '[a-zA-Z0-9_\-\.\+]+');
```

## 5.4 完整路由表

| 方法     | 路径                          | 说明           |
|--------|-----------------------------|--------------|
| GET    | /api/v1/health              | 健康检查         |
| POST   | /api/v1/login               | 用户登录         |
| POST   | /api/v1/register            | 用户注册         |
| POST   | /api/v1/logout              | 用户登出         |
| GET    | /api/v1/users               | 用户列表（分页）     |
| GET    | /api/v1/users/{id}          | 用户详情         |
| POST   | /api/v1/users               | 创建用户         |
| PUT    | /api/v1/users/{id}          | 更新用户         |
| DELETE | /api/v1/users/{id}          | 删除用户         |
| GET    | /api/v1/posts               | 文章列表（分页）     |
| GET    | /api/v1/posts/{id}          | 文章详情         |
| POST   | /api/v1/posts               | 创建文章         |
| PUT    | /api/v1/posts/{id}          | 更新文章         |
| DELETE | /api/v1/posts/{id}          | 删除文章         |
| GET    | /api/v1/posts/{id}/comments | 文章评论列表       |
| POST   | /api/v1/posts/{id}/comments | 添加评论         |
| GET    | /api/v1/stats               | 统计信息         |
| GET    | /api/v1/categories          | 分类列表         |
| POST   | /api/v1/categories          | 创建分类         |
| PUT    | /api/v1/categories/{id}     | 更新分类         |
| DELETE | /api/v1/categories/{id}     | 删除分类         |
| GET    | /api/v1/tags                | 标签列表         |
| POST   | /api/v1/tags                | 创建标签         |
| GET    | /api/v1/search              | 全局搜索         |
| GET    | /api/v1/upload/image        | 上传图片页面       |
| POST   | /api/v1/upload/image        | 上传图片         |
| POST   | /api/v1/upload/file         | 上传文件         |
| GET    | /api/v1/cache/test          | 缓存测试         |
| GET    | /api/v1/sql                 | 原生SQL查询      |
| POST   | /api/v1/transaction         | 事务测试         |
| GET    | /api/v1/pandadb-demo        | PandaDB双风格演示 |
| GET    | /api/v1/config              | 获取配置信息       |

| 方法   | 路径                      | 说明     |
|------|-------------------------|--------|
| POST | /api/v1/email/send      | 发送邮件   |
| POST | /api/v1/sms/send        | 发送短信   |
| GET  | /api/v1/logs            | 操作日志   |
| GET  | /api/v1/profile         | 当前用户信息 |
| PUT  | /api/v1/profile         | 更新个人信息 |
| PUT  | /api/v1/password        | 修改密码   |
| POST | /api/v1/forgot-password | 忘记密码   |

# 第6章 验证器

## 6.1 验证规则

| 规则            | 说明       | 示例                                  |
|---------------|----------|-------------------------------------|
| required      | 字段必填     | 'name' => 'required'                |
| string        | 必须是字符串   | 'name' => 'string'                  |
| integer       | 必须是整数    | 'age' => 'integer'                  |
| numeric       | 必须是数字    | 'price' => 'numeric'                |
| email         | 必须是邮箱格式  | 'email' => 'email'                  |
| url           | 必须是URL格式 | 'website' => 'url'                  |
| min:n         | 最小长度/值   | 'name' => 'min:2'                   |
| max:n         | 最大长度/值   | 'name' => 'max:50'                  |
| between:n,m   | 介于n和m之间  | 'age' => 'between:18,60'            |
| in:a,b,c      | 必须在列表中   | 'status' => 'in:0,1,2'              |
| not_in:a,b    | 不能在列表中   | 'role' => 'not_in:admin'            |
| regex:pattern | 正则匹配     | 'phone' => 'regex:/^1[3-9]\d{9}\$/' |
| confirmed     | 需要确认字段   | 'password' => 'confirmed'           |
| date          | 必须是日期格式  | 'birthday' => 'date'                |
| array         | 必须是数组    | 'ids' => 'array'                    |
| alpha         | 只能是字母    | 'name' => 'alpha'                   |
| alpha_num     | 字母和数字    | 'username' => 'alpha_num'           |

## 6.2 使用示例

验证器使用示例

```
<?php
use PandaAPI\Validation\Validator;
use PandaAPI\Core\Request;

// 基本验证
$request = new Request();
$data = $request->all();

$validator = new Validator($data, [
    'name'      => 'required|string|min:2|max:50',
    'email'     => 'required|email',
    'password'  => 'required|string|min:6|confirmed',
    'age'       => 'integer|between:18,120',
    'status'    => 'in:0,1,2',
]);
```

```

// 检查验证是否失败
if ($validator->fails()) {
    return Response::json([
        'error' => '!!!!',
        'messages' => $validator->errors(),
    ], 422);
}

// 获取验证后的安全数据
$safeData = $validator->validated();

// 自定义错误消息
$validator = new Validator($data, [
    'name' => 'required|string|min:2',
    'email' => 'required|email',
], [
    'name.required' => '!!!!!!',
    'name.min' => '!!!!2!!!',
    'email.required' => '!!!!!!',
    'email.email' => '!!!!!!!!!!',
]);

// 验证用户注册
$validator = new Validator($data, [
    'name' => 'required|string|min:2|max:50',
    'email' => 'required|email',
    'password' => 'required|string|min:6',
    'password_confirmation' => 'required|string|min:6',
    'phone' => 'regex:/^1[3-9]\d{9}$/',
]);

if ($validator->fails()) {
    return Response::json([
        'code' => 422,
        'message' => '!!!!!!',
        'errors' => $validator->errors(),
    ], 422);
}

```

# 第7章 中间件

## 7.1 内置中间件

| 中间件      | 说明         | 配置参数                        |
|----------|------------|-----------------------------|
| Cors     | 跨域资源共享     | origin, methods, headers    |
| Auth     | Token 认证   | token 请求头或参数                |
| Throttle | 接口限流       | max_attempts, decay_minutes |
| Log      | 请求日志记录     | 日志文件路径                      |
| JsonBody | JSON 请求体解析 | 自动解析 JSON 请求                |

## 7.2 使用示例

中间件使用示例

```
<?php
use PandaAPI\Route\Route;

// 全局中间件 - 在 App 配置中设置
$app->config([
    'middleware' => ['Cors', 'Log', 'JsonBody'],
]);

// 路由组中间件
Route::group(['middleware' => ['Cors']], function() {
    Route::get('/api/public', function() {
        return Response::success(['message' => 'oooo']);
    });
});

// 单个路由中间件
Route::get('/api/profile', function() {
    return Response::success(['user' => 'ooooo']);
})->middleware(['Auth']);

// 限流中间件 (60次/分钟)
Route::post('/api/sms/send', function() {
    return Response::success(['message' => 'oooo']);
})->middleware(['Throttle:60,1']);

// 组合中间件
Route::post('/api/admin/settings', function() {
    return Response::success(['message' => 'oooo']);
})->middleware(['Auth', 'Throttle:30,1']);

// 自定义中间件示例
class CheckAdmin
{
    public function handle($request, Closure $next)
    {
        $user = $request->getAttribute('user');
        if ($user['role'] !== 'admin') {
```

```
        return Response::error('!!!!', 403);
    }
    return $next($request);
}
}

// 注册自定义中间件
Route::get('/api/admin/dashboard', function() {
    return Response::success(['stats' => '...']);
})->middleware(['Auth', 'CheckAdmin']);
```



# 第8章 公共函数

## 8.1 函数列表

| 函数名               | 说明            | 参数                    |
|-------------------|---------------|-----------------------|
| json_success      | 返回成功JSON响应    | data, message, code   |
| json_error        | 返回错误JSON响应    | message, code, data   |
| get_input         | 获取输入参数（支持默认值） | key, default          |
| generate_token    | 生成随机Token     | length                |
| password_encrypt  | 密码加密          | password              |
| password_verify   | 密码验证          | password, hash        |
| get_user_id       | 获取当前登录用户ID    | request               |
| get_client_ip     | 获取客户端IP       | request               |
| format_date       | 格式化日期         | timestamp, format     |
| pagination        | 生成分页数据        | total, page, per_page |
| array_to_tree     | 数组转树形结构       | data, pid, id         |
| curl_get          | CURL GET请求    | url, headers, timeout |
| curl_post         | CURL POST请求   | url, data, headers    |
| is_mobile         | 判断是否手机访问      | user_agent            |
| generate_order_no | 生成订单号         | prefix                |
| upload_file       | 文件上传处理        | file, path            |
| send_sms          | 发送短信          | phone, content        |
| send_email        | 发送邮件          | to, subject, body     |
| write_log         | 写入日志          | message, level, file  |
| config            | 获取配置项         | key, default          |
| env               | 获取环境变量        | key, default          |
| redirect          | 重定向           | url                   |
| abort             | 中止请求返回错误      | code, message         |

## 8.2 使用示例

公共函数使用示例

```
<?php
// json_success - 成功响应
return json_success(['id' => 1, 'name' => '张三']);
// 输出: {"code": 200, "message": "success", "data": {"id": 1, "name": "张三"}}
```

```

// json_error - 错误响应
return json_error('####', 404);
// 输出: {"code": 404, "message": "用户不存在", "data": null}

// get_input - 获取输入参数
$name = get_input('name', ''); // POST/GET 参数
$email = get_input('email', 'default'); // 带默认值

// generate_token - 生成Token
$token = generate_token(32);
// 输出: "a1b2c3d4e5f6..."

// password_encrypt / password_verify
$hash = password_encrypt('123456');
$isValid = password_verify('123456', $hash); // true

// get_client_ip - 获取客户端IP
$ip = get_client_ip($request);
// 输出: "192.168.1.100"

// curl_get - GET请求
$data = curl_get('https://api.example.com/users', [
    'Authorization: Bearer ' . $token,
]);

// curl_post - POST请求
$result = curl_post('https://api.example.com/users', [
    'name' => 'aa',
    'email' => 'zhangsan@example.com',
], ['Content-Type: application/json']);

// write_log - 写日志
write_log('#####: ' . $email, 'info');
write_log('#####: ' . $e->getMessage(), 'error');

// pagination - 分页数据
$pageData = pagination(100, 1, 15);
// 返回: ['total' => 100, 'current_page' => 1, 'per_page' => 15, 'last_page' => 7]

```

## 第9章 MVC 应用开发

### 9.1 控制器 Controller

UserController 控制器

```
<?php
namespace App\Controllers;

use PandaAPI\Core\Request;
use PandaAPI\Core\Response;
use PandaAPI\Database\PandaDB;
use PandaAPI\Cache\Cache;
use PandaAPI\Validation\Validator;

class UserController
{
    /**
     * 用户列表 GET /users
     */
    public function index(Request $request)
    {
        $page = (int)$request->query('page', 1);
        $perPage = (int)$request->query('per_page', 15);
        $status = $request->query('status');

        $builder = PandaDB::table('users');
        if ($status !== null) {
            $builder->where('status', (int)$status);
        }

        $result = $builder->order('created_at', 'desc')
            ->paginate($perPage, $page);

        return Response::paginate(
            $result['data'],
            $result['total'],
            $result['current_page'],
            $result['per_page']
        );
    }

    /**
     * 用户详情 GET /users/{id}
     */
    public function show(Request $request, $id)
    {
        $user = Cache::remember("user:{$id}", function() use ($id) {
            return PandaDB::table('users')
                ->where('id', (int)$id)
                ->find();
        }, 300);

        if (!$user) {
            return Response::error('用户不存在', 404);
        }

        unset($user['password']);
        return Response::success($user);
    }
}
```

```

    }

/**
 * 创建用户 POST /users
 */
public function store(Request $request)
{
    $data = $request->all();

    $validator = new Validator($data, [
        'name'      => 'required|string|min:2|max:50',
        'email'     => 'required|email',
        'password' => 'required|string|min:6',
    ]);

    if ($validator->fails()) {
        return Response::json([
            'error'      => '失败',
            'messages' => $validator->errors()
        ], 422);
    }

    $data['password'] = password_hash($data['password'], PASSWORD_DEFAULT);
    $data['status']   = 1;
    $data['created_at'] = date('Y-m-d H:i:s');

    $id = PandaDB::table('users')->insert($data);
    $user = PandaDB::table('users')->where('id', $id)->find();
    unset($user['password']);

    return Response::json($user, 201);
}
}

```

## 9.2 模型 Model

### UserModel 模型

```

<?php
namespace App\Models;

use PandaAPI\Database\PandaDB;

class UserModel
{
    // 表名 (不含前缀)
    protected $table = 'users';

    // 获取所有用户
    public function getAll($status = null)
    {
        $builder = PandaDB::table($this->table);
        if ($status !== null) {
            $builder->where('status', (int)$status);
        }
        return $builder->order('created_at', 'desc')->select();
    }

    // 根据ID查找用户
    public function findById($id)
    {

```

```

    {
        return PandaDB::table($this->table)
            ->where('id', (int)$id)
            ->find();
    }

// 根据邮箱查找
public function findByEmail($email)
{
    return PandaDB::table($this->table)
        ->where('email', $email)
        ->find();
}

// 创建用户
public function create(array $data)
{
    $data['status'] = 1;
    $data['created_at'] = date('Y-m-d H:i:s');
    return PandaDB::table($this->table)->insert($data);
}

// 更新用户
public function update($id, array $data)
{
    $data['updated_at'] = date('Y-m-d H:i:s');
    return PandaDB::table($this->table)
        ->where('id', (int)$id)
        ->update($data);
}

// 删除用户
public function delete($id)
{
    return PandaDB::table($this->table)
        ->where('id', (int)$id)
        ->delete();
}

// 分页查询
public function paginate($perPage = 15, $page = 1)
{
    return PandaDB::table($this->table)
        ->order('created_at', 'desc')
        ->paginate($perPage, $page);
}
}

```

### 9.3 公共函数 common

#### 公共函数 common.php

```

<?php
// app/common.php

/**
 * 返回成功JSON响应
 */
function json_success($data = null, $message = 'success', $code = 200)
{

```

```

        return json_encode([
            'code' => $code,
            'message' => $message,
            'data' => $data,
        ], JSON_UNESCAPED_UNICODE);
    }

/**
 * 返回错误JSON响应
 */
function json_error($message = 'error', $code = 400, $data = null)
{
    return json_encode([
        'code' => $code,
        'message' => $message,
        'data' => $data,
    ], JSON_UNESCAPED_UNICODE);
}

/**
 * 获取输入参数
 */
function get_input($key = null, $default = null)
{
    $input = array_merge($_GET, $_POST);
    $jsonInput = json_decode(file_get_contents('php://input'), true);
    if (is_array($jsonInput)) {
        $input = array_merge($input, $jsonInput);
    }

    if ($key === null) {
        return $input;
    }
    return $input[$key] ?? $default;
}

/**
 * 生成随机Token
 */
function generate_token($length = 32)
{
    return bin2hex(random_bytes($length));
}

/**
 * 密码加密
 */
function password_encrypt($password)
{
    return password_hash($password, PASSWORD_DEFAULT);
}

/**
 * 获取客户端IP
 */
function get_client_ip()
{
    if (!empty($_SERVER['HTTP_X_FORWARDED_FOR'])) {
        $ips = explode(',', $_SERVER['HTTP_X_FORWARDED_FOR']);
        return trim($ips[0]);
    }
}

```

```
    return $_SERVER['REMOTE_ADDR'] ?? '0.0.0.0';
}
```

## 9.4 路由配置 router

路由配置 routes/api.php

```
<?php
// routes/api.php

use PandaAPI\Route\Route;
use App\Controllers\UserController;
use App\Controllers\PostController;
use App\Controllers\AuthController;

// 健康检查
Route::get('/api/v1/health', function() {
    return Response::success([
        'framework' => 'PandaAPI',
        'status'     => 'healthy',
        'time'       => date('Y-m-d H:i:s'),
    ]);
});

// 认证路由 (公开)
Route::post('/api/v1/login', [AuthController::class, 'login']);
Route::post('/api/v1/register', [AuthController::class, 'register']);

// API 路由组
Route::group([
    'prefix'      => '/api/v1',
    'middleware' => ['Cors', 'Auth'],
], function() {

    // 用户资源路由
    Route::get('/users', [UserController::class, 'index']);
    Route::get('/users/{id}', [UserController::class, 'show']);
    Route::post('/users', [UserController::class, 'store']);
    Route::put('/users/{id}', [UserController::class, 'update']);
    Route::delete('/users/{id}', [UserController::class, 'destroy']);

    // 文章资源路由
    Route::get('/posts', [PostController::class, 'index']);
    Route::get('/posts/{id}', [PostController::class, 'show']);
    Route::post('/posts', [PostController::class, 'store']);
    Route::put('/posts/{id}', [PostController::class, 'update']);
    Route::delete('/posts/{id}', [PostController::class, 'destroy']);

    // 统计接口
    Route::get('/stats', function() {
        return Response::success([
            'users' => PandaDB::table('users')->count(),
            'posts' => PandaDB::table('posts')->count(),
        ]);
    });
});
```

# 第10章 API 接口测试

## 10.1 curl 测试命令

### curl 接口测试命令

```
# =====
# 熊猫API框架 - 完整接口测试命令
# 基础地址: http://localhost:8080
# =====

# 1. 健康检查
curl -X GET http://localhost:8080/api/v1/health

# 2. 用户注册
curl -X POST http://localhost:8080/api/v1/register \
-H "Content-Type: application/json" \
-d '{"name":"", "email":"zhangsan@test.com", "password":"123456"}'

# 3. 用户登录
curl -X POST http://localhost:8080/api/v1/login \
-H "Content-Type: application/json" \
-d '{"email":"zhangsan@test.com", "password":"123456"}'

# 4. 获取用户列表 (分页)
curl -X GET "http://localhost:8080/api/v1/users?page=1&per_page=15"

# 5. 获取用户详情
curl -X GET http://localhost:8080/api/v1/users/1

# 6. 创建用户
curl -X POST http://localhost:8080/api/v1/users \
-H "Content-Type: application/json" \
-H "Authorization: Bearer YOUR_TOKEN" \
-d '{"name":"", "email":"lisi@test.com", "password":"123456"}'

# 7. 更新用户
curl -X PUT http://localhost:8080/api/v1/users/1 \
-H "Content-Type: application/json" \
-H "Authorization: Bearer YOUR_TOKEN" \
-d '{"name":""}'

# 8. 删除用户
curl -X DELETE http://localhost:8080/api/v1/users/1 \
-H "Authorization: Bearer YOUR_TOKEN"

# 9. 获取文章列表
curl -X GET "http://localhost:8080/api/v1/posts?page=1&per_page=10"

# 10. 创建文章
curl -X POST http://localhost:8080/api/v1/posts \
-H "Content-Type: application/json" \
-H "Authorization: Bearer YOUR_TOKEN" \
-d '{"title":"", "content":""}'

# 11. 获取统计信息
curl -X GET http://localhost:8080/api/v1/stats

# 12. 缓存测试
```



```
curl -X GET http://localhost:8080/api/v1/cache/test

# 13. 原生SQL查询
curl -X GET http://localhost:8080/api/v1/sql

# 14. 事务测试
curl -X POST http://localhost:8080/api/v1/transaction \
-H "Authorization: Bearer YOUR_TOKEN"

# 15. PandaDB 双风格演示
curl -X GET http://localhost:8080/api/v1/pandadb-demo

# 16. 全局搜索
curl -X GET "http://localhost:8080/api/v1/search?keyword=PHP"

# 17. 获取分类列表
curl -X GET http://localhost:8080/api/v1/categories

# 18. 获取当前用户信息
curl -X GET http://localhost:8080/api/v1/profile \
-H "Authorization: Bearer YOUR_TOKEN"

# 19. 修改密码
curl -X PUT http://localhost:8080/api/v1/password \
-H "Content-Type: application/json" \
-H "Authorization: Bearer YOUR_TOKEN" \
-d '{"old_password":"123456","new_password":"654321"}'

# 20. 上传图片
curl -X POST http://localhost:8080/api/v1/upload/image \
-H "Authorization: Bearer YOUR_TOKEN" \
-F "image=@/path/to/image.jpg"
```

— 文档结束 —

熊猫API框架 PandaAPI Framework v1.0